# API Security Risk Analysis Report

Prepared by: NISHANTH N NAIK

Date: 11 May 2026

CIN ID: FIT/APR26/CS7733

# 1. Introduction

Application Programming Interfaces (APIs) allow software applications to communicate and exchange data over networks. APIs are widely used in web applications, mobile applications, and cloud services for accessing and managing data. Since APIs often expose sensitive information and business functionality, improper security controls can lead to unauthorized access, data leakage, and account compromise.

This report analyzes common API security risks identified during testing of publicly available demo APIs using Postman.

# 2. Scope

APIs Tested

- JSONPlaceholder API
- ReqRes API

Tools Used

Postman Official Website

## Testing Methodology

The assessment was performed using a read-only and ethical testing approach. Testing activities included:

- Sending GET and POST requests
- Inspecting API responses
- Reviewing HTTP response headers
- Testing authentication behavior
- Verifying access control mechanisms

No destructive testing or exploitation was performed.

## 3.Tools Used

The following tools and platforms were used during the API security assessment:

| Tool | Purpose |
| --- | --- |
| Postman | API testing and request analysis |
| Canva Official Website | Report design and formatting |
| GitHub Official Website | Repository hosting and project submission |
| JSONPlaceholder API | Demo API used for testing |
| Tool | Purpose |

## 4. Risk Findings Table

| # | Risk Found | Severity | Screenshot |
|---|---|---|---|
| 1 | No authentication on /users | 🔴 High | Figure 1 |
| 2 | Excessive data exposure | 🔴 High | Figure 2 |
| 3 | IDOR – access any user by ID | 🔴 High | Figure 3,4 |
| 4 | Missing security headers | 🟡 Medium | Figure 5,6,7 |
| 5 | Unauthenticated POST allowed | 🟡 Medium | Figure 8 |
| 6 | Weak token / no expiry visible | 🟡 Medium | Figure 9 |

## 5. Detailed Findings

## Finding 1 — No Authentication on /users

Severity: High

Description: The API endpoint /users returned complete user information without requiring any authentication or authorization token. Any user can access the endpoint directly using a browser or API testing tool.
This increases the risk of unauthorized data access and user enumeration in production environments.

Evidence
Endpoint tested: GET https://jsonplaceholder.typicode.com/users

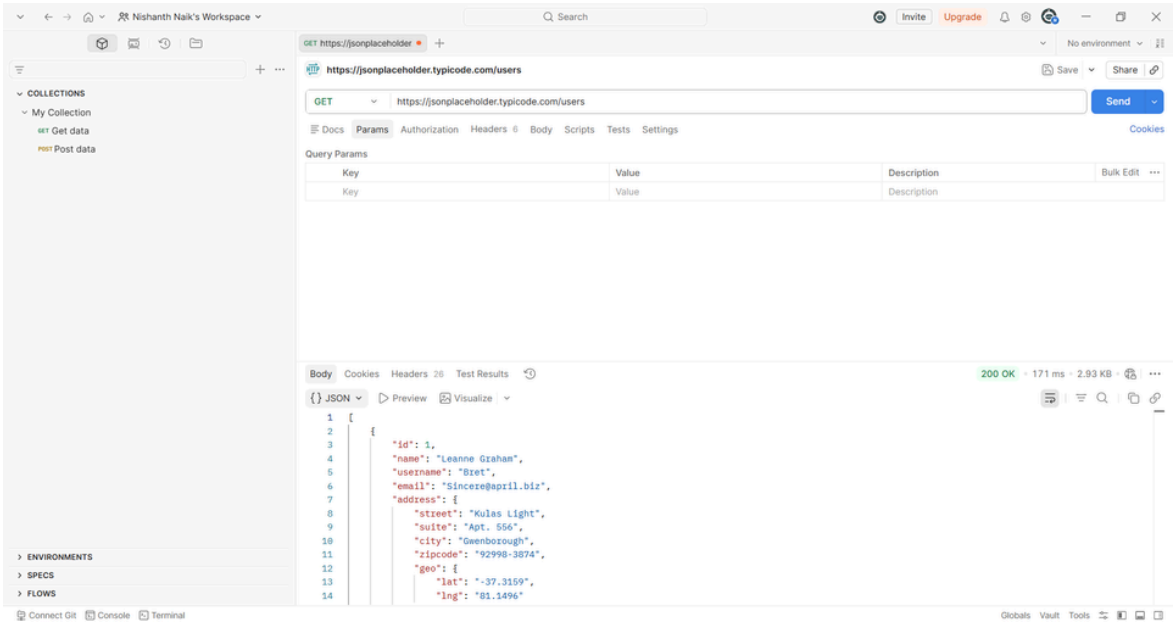


Figure 1: GET request to /users returning user data without authentication

## Remediation

- Enforce authentication using JWT or OAuth 2.0
- Apply role-based access controls
- Restrict public access to sensitive endpoints

## Finding 2 — Excessive Data Exposure

Severity:High

Description:The API response exposed unnecessary user information including email addresses, geographic coordinates, and address details. APIs should only expose data that is strictly required by the client application.

Excessive data exposure may assist attackers during reconnaissance and social engineering attacks.

Evidence
Endpoint tested: GET https://jsonplaceholder.typicode.com/users/1

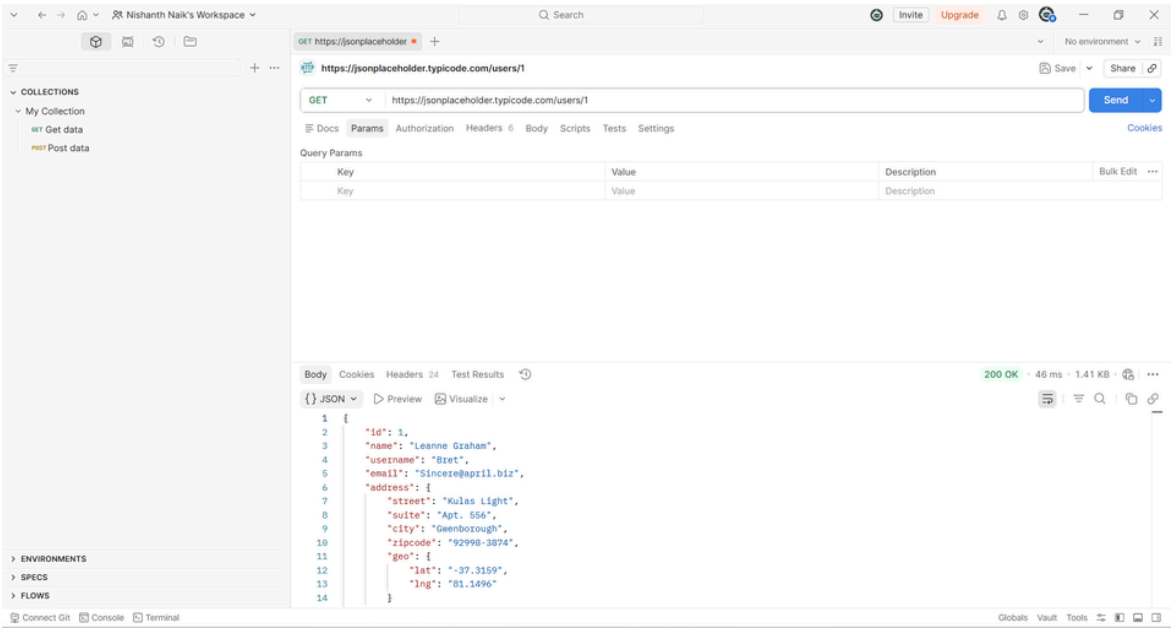


Figure 2: Sensitive user information exposed in API response

## Remediation

- Limit exposed response fields
- Implement response filtering
- Follow the principle of least privilege

## Finding 3 — IDOR (Insecure Direct Object Reference)

Severity: High

Description: The API allowed direct access to different user records simply by changing the numeric user ID in the URL.

Example:
GET /users/1
GET /users/2
GET /users/3

This behavior demonstrates a potential IDOR vulnerability where attackers may enumerate and access unauthorized records.

Evidence

Endpoints tested:

- GET https://jsonplaceholder.typicode.com/users/2
- GET https://jsonplaceholder.typicode.com/users/3

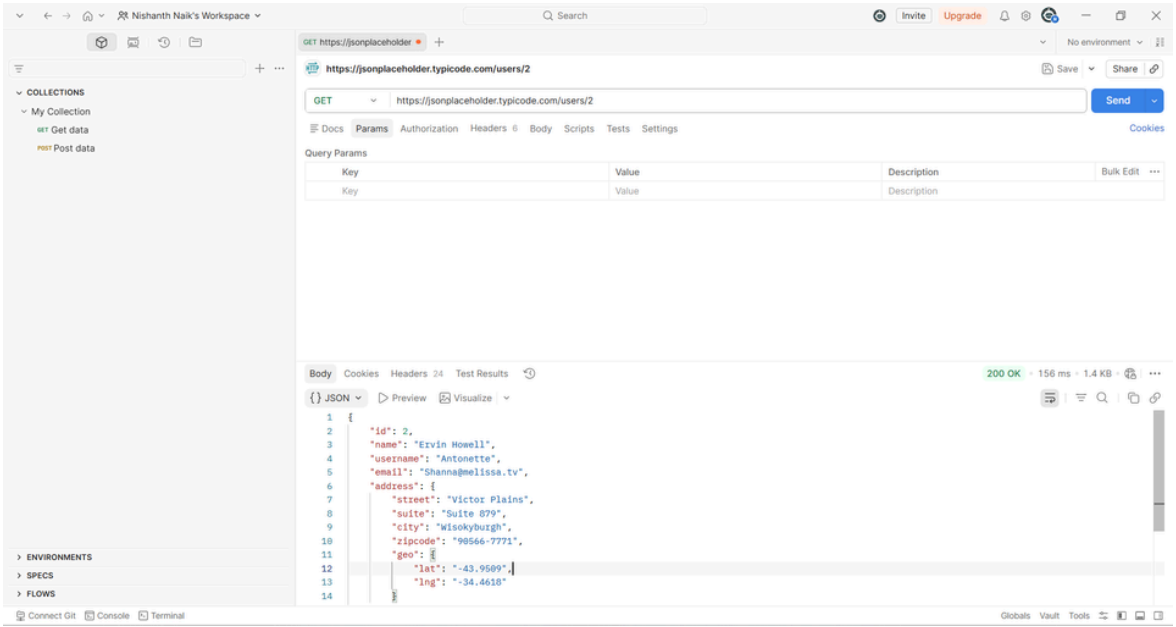

Figure 3: Accessing different user records by modifying URL ID values

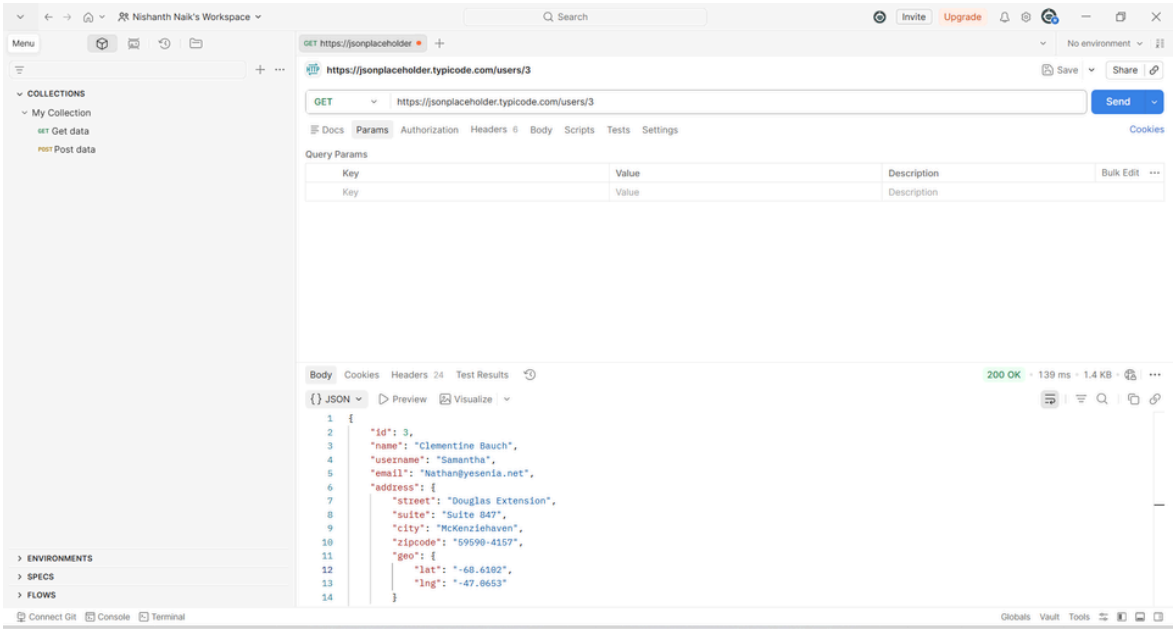

Figure 4: Accessing different user records by modifying URL ID values

## Remediation

- Validate object ownership server-side
- Enforce authorization checks
- Use indirect reference identifiers where possible

## Finding 4 — Missing Security Headers

Severity: Medium

Description:
The response headers lacked several commonly recommended security headers such as:

- Content-Security-Policy
- X-Frame-Options
- Strict-Transport-Security

Missing security headers may increase exposure to clickjacking, MIME sniffing, and browser-based attacks.

Evidence: Headers reviewed in Postman response.

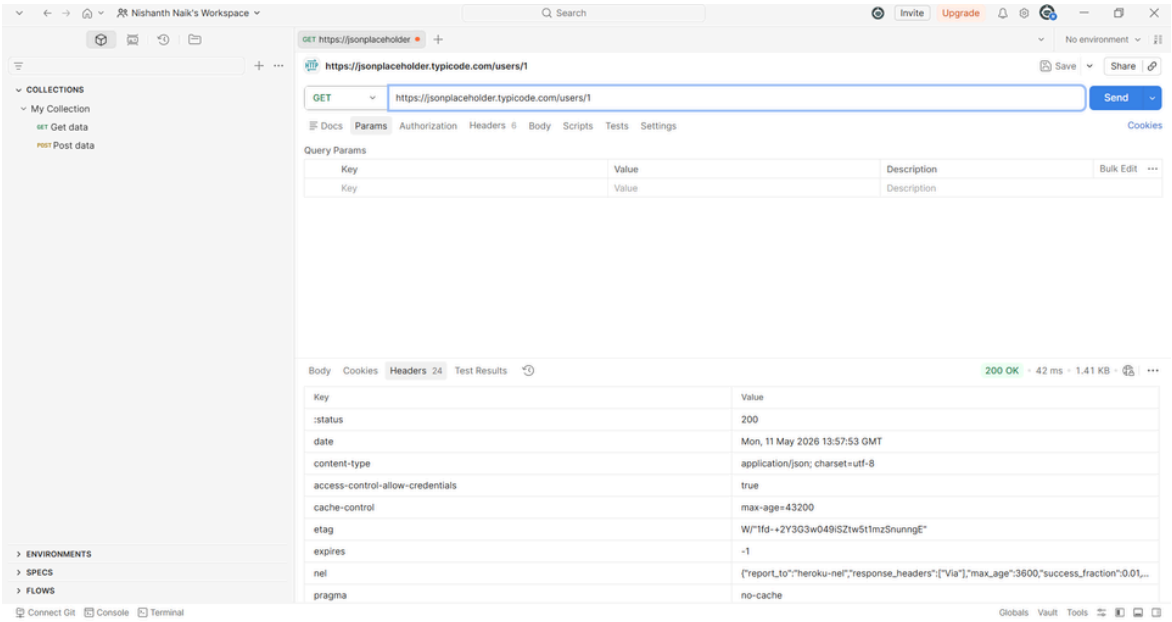


Figure 5: Missing recommended HTTP security headers

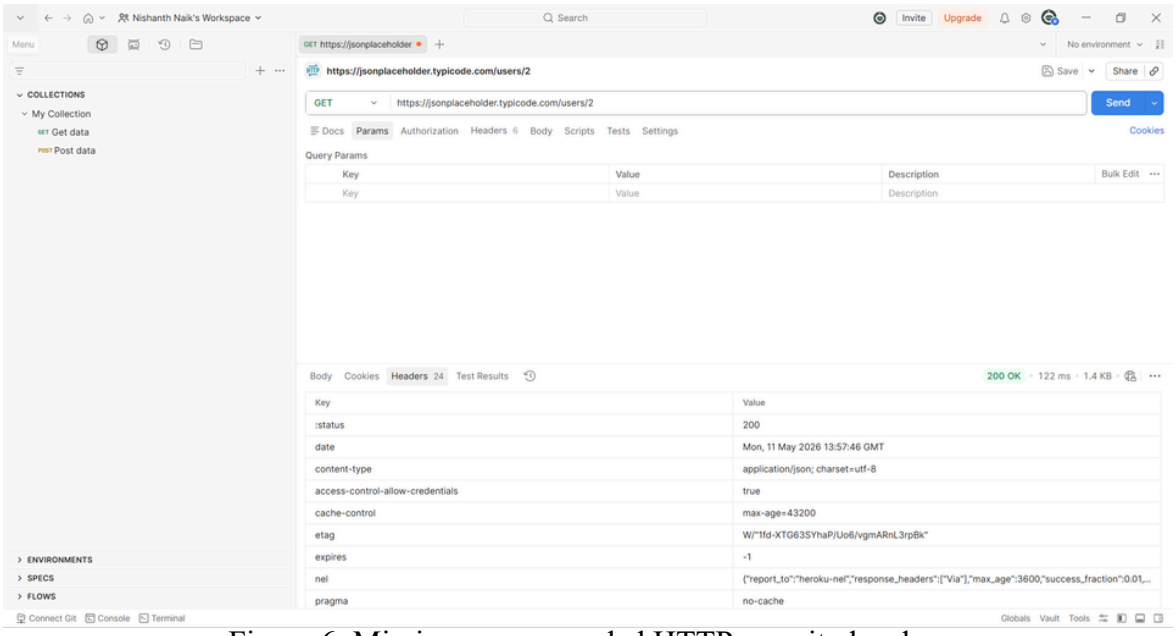


Figure 6: Missing recommended HTTP security headers

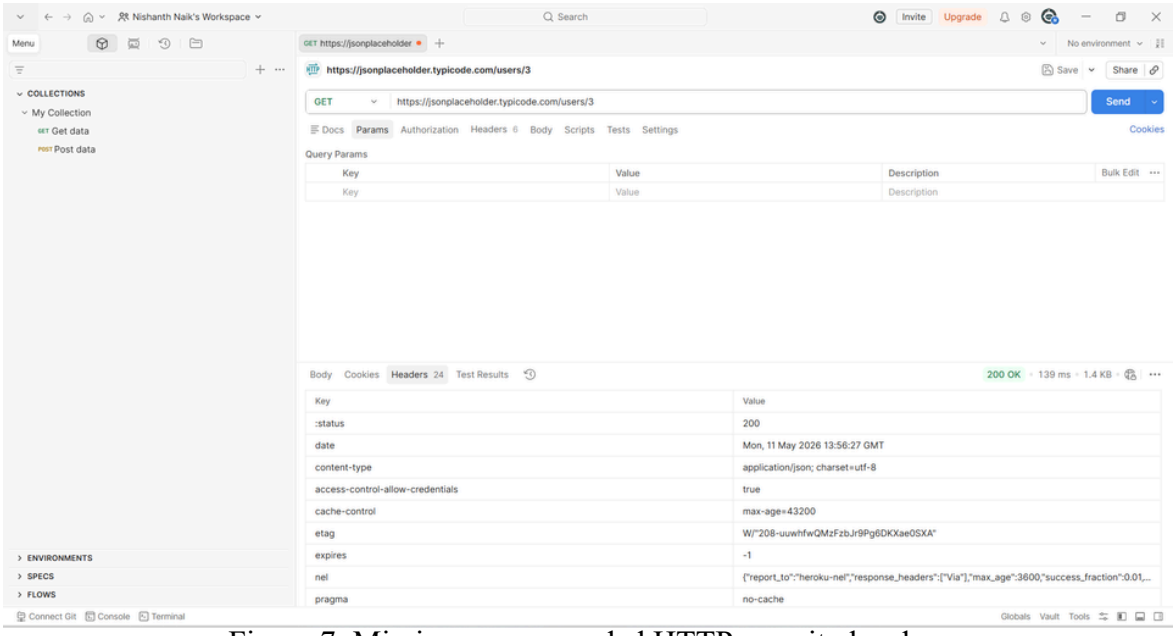


Figure 7: Missing recommended HTTP security headers

# Remediation

- Configure secure HTTP headers
- Implement HSTS
- Add Content Security Policy (CSP)

# Finding 5 — Unauthenticated POST Allowed

Severity: Medium

Description:
The API accepted POST requests without authentication credentials and returned a successful response. This demonstrates weak access control for resource creation functionality.

Evidence
Endpoint tested: POST https://jsonplaceholder.typicode.com/posts

Payload:

```
{
 "title": "Security Test",
 "body": "This was posted without any login",
 "userId": 1
}
```

Response: 201 Created

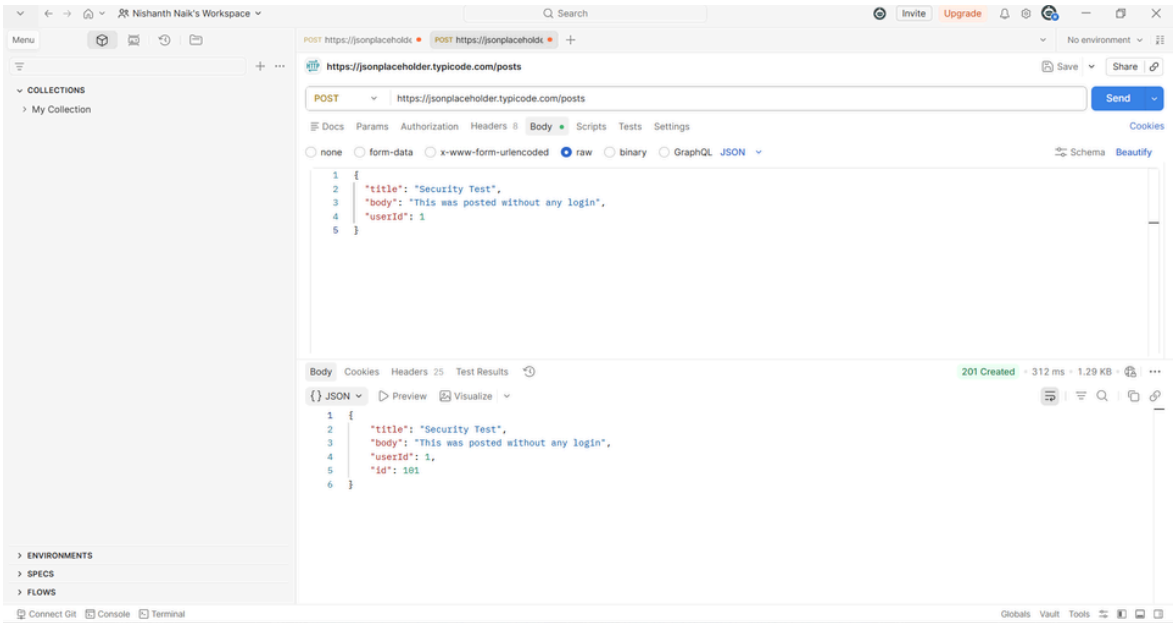

Figure 8 POST request accepted without authentication

## Remediation

- Require authentication for POST operations
- Validate authorization before resource creation
- Implement request logging and monitoring

## Finding 6 — Weak Token / No Expiry Visible

Severity: Medium

Description:
The login endpoint returned a token after authentication, but no token expiration information or additional protection mechanisms were visible.
Weak token management can increase the risk of session hijacking and unauthorized reuse of tokens.

Evidence
Endpoint tested: POST https://reqres.in/api/login

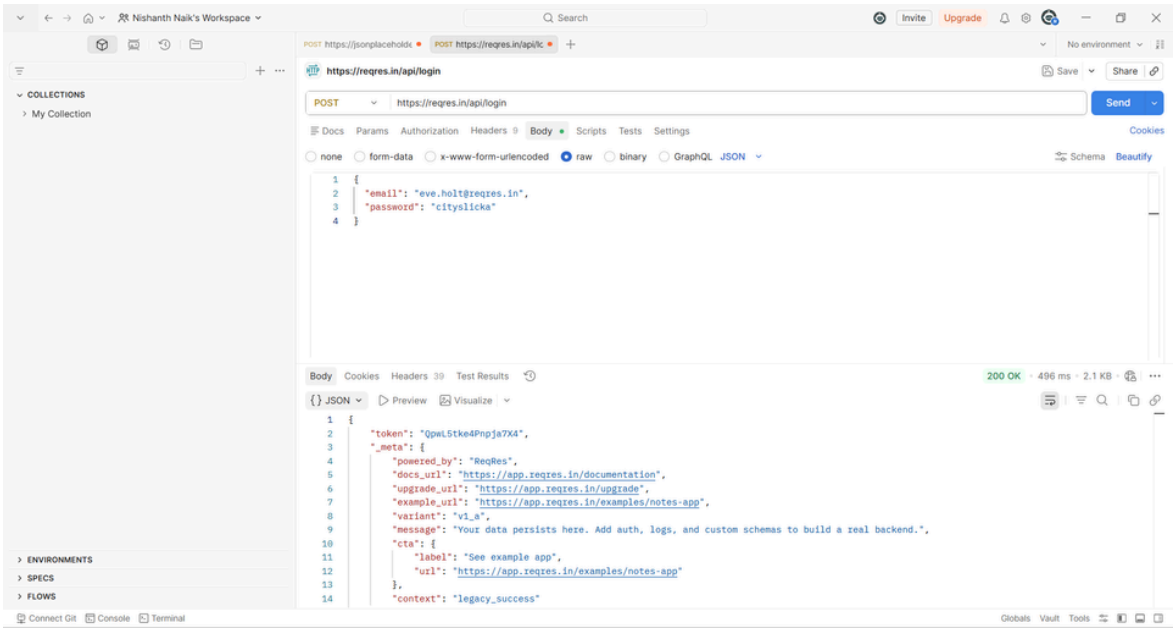


Figure 9: Authentication token returned after login request

### Remediation

- Use short-lived JWT tokens
- Implement token expiration and refresh policies
- Use HTTPS for all token transmission

## 6. Remediation Summary

- Enforce authentication on sensitive endpoints
- Apply authorization checks for all object access
- Restrict unnecessary data exposure
- Implement secure HTTP security headers
- Require authentication for POST and PUT requests
- Use secure token expiration policies
- Monitor and log suspicious API activity
- Apply rate limiting and abuse prevention controls

## 7.Conclusion

The API security assessment identified several common API security weaknesses including missing authentication, excessive data exposure, IDOR behavior, and weak token handling practices. While the tested APIs are intentionally public demo services, these findings demonstrate how similar issues in production systems could lead to unauthorized access and sensitive data exposure.

Proper authentication, authorization, secure headers, and least-privilege data exposure should be implemented to reduce API security risks effectively.